



SRS: SMART ROAD SYSTEM

Progetto di SCALABLE AND RELIABLE SERVICES

Smart Road System

Gestione resiliente e ottimizzazione intelligente del traffico

Autore:
Andrea La Rocca, Cristina
Tomaciello

Anno Accademico:
2025/2026

Indice

1	Introduzione e Definizione del Servizio	2
1.1	Dominio e Contesto del Servizio	2
1.2	Stakeholder e Utenti Target	2
1.3	Carico di Lavoro ed Eventi Principali	2
2	Architettura Core	4
2.1	Progettazione tramite l'Availability Cube	4
2.2	L'Edge Layer e la Graceful Degradation	5
2.3	Il Communication Layer	5
2.4	Il Persistence Layer, Disaccoppiamento e Resilienza Infrastrutturale	6
2.5	Il Control Layer: Watchdog, Quorum e Prevenzione Split-Brain	7
3	Architettura IA: Livello Agentico, Protocollo MCP e Osservabilità	8
3.1	Il Bridge Service e la Sincronizzazione Distribuita	8
3.2	Z-Axis Scalability e l'Isolamento dell'IA	9
3.2.1	Opencode Brain e Architettura Gerarchica degli Agenti	9
3.2.2	Guardrails e Model Context Protocol (MCP)	10
3.2.3	Workflow del Traffico e Recovery Hardware (Sense-Think-Act)	10
3.2.4	Protezione OPEX: Il parametro <i>Steps</i> e la Verbosity	11
3.3	Osservabilità e Asimmetria nei Trade-off Economici (80/20 Rule)	11
4	Policy di Sicurezza Operativa e Guardrails per l'Intelligenza Artificiale	12
4.1	Principio del Least Privilege e Bounded Execution	12
4.2	Policy-as-Code e Loop Prevention	12
4.3	Mitigazione dell'Agent Drift e delle Allucinazioni	12
4.4	Operational Safety Policy e Gestione dell'Ambiguità	13
5	Validazione e Testing del Sistema	14
5.1	Metodologia e Strategia di Testing	14
5.2	Stress Test del Communication Layer (MQTT Flooding)	14
5.2.1	Risultati Empirici a 10.000 msg/sec	14
5.2.2	Analisi Comparativa: Il Costo della Certezza (QoS 0 vs QoS 1)	14
5.3	Chaos Engineering: Validazione dell'Immunità ai Guasti	15
5.3.1	Matrice dei Test di Fallimento (Node Failure Matrix)	15
5.3.2	MTTR: Componente vs. Sistema	16
5.3.3	Mean Time To Detect (MTTD) e il Trade-off del Watchdog	16
6	Analisi Economica (OPEX), Trade-off Ingegneristici e Return on Agent (ROA)	17
6.1	Analisi del Payload e Return on Agent (ROA)	17
6.2	Limiti Economici e Protezione del Budget (Cost Ceiling)	17

Capitolo 1

Introduzione e Definizione del Servizio

1.1 Dominio e Contesto del Servizio

Nell'ambito delle moderne infrastrutture urbane la gestione della viabilità richiede sistemi in grado di operare in modo continuo, reattivo e sicuro anche in presenza di guasti, picchi di carico e incertezza operativa. Per rispondere a queste esigenze, il presente progetto si colloca nel dominio dell'Industrial / IoT Monitoring e propone la progettazione e l'implementazione di **Smart Road System (SRS)**, un servizio distribuito per il monitoraggio e l'ottimizzazione automatizzata di una rete semaforica cittadina. SRS sfrutta un'architettura a microservizi basata su container e integra un livello operativo supportato da Intelligenza Artificiale Agentica per assistere la gestione dei guasti e l'ottimizzazione dei flussi.

1.2 Stakeholder e Utenti Target

Il sistema SRS è stato progettato tenendo in considerazione tre categorie principali di stakeholder:

- **Autorità Cittadine e Ingegneri del Traffico:** Necessitano di una piattaforma di observability centralizzata per monitorare lo stato di salute degli incroci e delegare ottimizzazioni sicure in tempo reale.
- **Servizi di Emergenza e Manutenzione:** Richiedono un sistema di allerta immediato in caso di guasti hardware o software dei semafori, per intervenire tempestivamente.
- **Cittadini e Pendolari:** Gli utenti finali del sistema, il cui obiettivo è una riduzione dei tempi di percorrenza e la garanzia della sicurezza stradale in ogni condizione.

1.3 Carico di Lavoro ed Eventi Principali

Il sistema gestisce un carico di lavoro continuo e altamente asincrono, strutturato principalmente su due flussi di eventi:

- **Ingestione della Telemetria:** I nodi semaforici pubblicano costantemente lo stato delle luci e il conteggio dei veicoli in coda su topic MQTT dedicati.
- **Gestione degli Allarmi e Comandi:** Viene analizzata la telemetria per generare eventi critici come allarmi di congestione (es. superamento della soglia di 15 auto) o allarmi di silenzio hardware (es. nodo offline da oltre 30 secondi). A seguito dell'elaborazione, il sistema invia comandi di ripristino o nuove configurazioni di temporizzazione ai nodi interessati.

1.4 Criticità e Scenari di Guasto

Nel contesto della mobilità urbana, il progetto Smart Road System (SRS) si configura come un vero e proprio Cyber-Physical System (CPS). La Reliability del sistema cessa di essere una metrica puramente tecnica per diventare un requisito critico per la sicurezza pubblica: un guasto non gestito a un incrocio genera incidenti, congestioni a cascata e ritarda l'intervento dei mezzi di soccorso.

La resilienza assume come principio di base il fatto che sia impossibile evitare del tutto tali incidenti, specialmente in reti distribuite su larga scala. L'architettura SRS è stata progettata per tollerare e mitigare tre macro-categorie di scenari di guasto realistici:

Guasti di Sistema: nel nostro dominio, questi si traducono in:

- **Degrado Hardware all’Edge:** I nodi fisici all’incrocio sono esposti a usura, sbalzi di tensione o guasti ai sensori.
- **Colli di bottiglia e Partizioni di Rete:** Saturazione improvvisa della rete o cadute dei nodi broker MQTT che interrompono la comunicazione tra l’incrocio (Edge) e il cervello centrale (Control/AI Layer), lasciando i semafori isolati.
- **Crash del Software e dei Database:** Corruzione della memoria nei microservizi o indisponibilità improvvisa dei database time-series (InfluxDB), essenziali per le logiche dell’intelligenza artificiale.

Errori Umani: Sebbene spesso sottovalutato, il fattore umano è la seconda causa principale di disservizio. Le persone sono intrinsecamente fallibili, soprattutto se sottoposte a pressione operativa. In SRS, un errore umano potrebbe consistere in un aggiornamento software errato spinto in produzione (bug introdotto in un nodo semaforico) o in una configurazione (policy-as-code) mal scritta per l’agente IA, che potrebbe portare a cicli di verde non sicuri. Il sistema deve adottare un approccio sistemico inserendo “guardrails” e barriere difensive per limitare preventivamente il raggio d’azione degli operatori e degli agenti autonomi.

Eventi Esterni Accidentali: Fenomeni fuori dal controllo organizzativo, come blackout elettrici prolungati a livello di quartiere, allagamenti o condizioni meteorologiche estreme che danneggiano fisicamente le infrastrutture di comunicazione cittadine.

A fronte di questo spettro di minacce, l’obiettivo architetturale di SRS non è la creazione di un sistema utopicamente perfetto, ma l’implementazione pratica del modello di resilienza. In ogni scenario sopra descritto, il sistema deve garantire un rilevamento istantaneo dell’anomalia (Detection), l’isolamento del raggio del guasto (Blast Radius) e l’attivazione tempestiva di meccanismi di Graceful Degradation, in cui l’incrocio rinuncia all’ottimizzazione globale (passando ad esempio al giallo lampeggiante) ma preserva la sicurezza vitale per i cittadini.

Capitolo 2

Architettura Core

2.1 Progettazione tramite l'Availability Cube

L'intera topologia di **Smart Road System** è stata ingegnerizzata mappando i requisiti fisici e logici sulle tre dimensioni dell'**Availability Cube** (noto anche come *AKF Scale Cube*). L'obiettivo è stato quello di bilanciare la scalabilità con la resilienza, accettando consapevolmente i compromessi necessari tra prestazioni e coerenza.

- **Asse X (Replicazione Orizzontale e Ridondanza):** Per eliminare ogni *Single Point of Failure* (SPoF) infrastrutturale i componenti critici sono stati replicati. Il sistema implementa un cluster MQTT **Active-Active** tramite nodi EMQX bilanciati da HAProxy, garantendo la continuità del *Communication Layer*. La persistenza dei dati abbandona i fragili pattern applicativi in favore di un disaccoppiamento totale, la scrittura su InfluxDB (nodi Alpha e Beta) è demandata a due nodi Telegraf operanti in modalità Active-Active, mentre un cluster HAProxy dedicato scherma i database gestendo il failover trasparente delle letture. Il consenso distribuito è invece garantito da un quorum su tre nodi Redis, privilegiando la Strong Consistency per le operazioni di Leader Election del Main Controller.
- **Asse Y (Scomposizione Funzionale e Disaccoppiamento):** SRS adotta una netta separazione tra il **Data Plane** (Edge) e il **Control Plane** (Controller e IA). Il disaccoppiamento è garantito dall'eliminazione delle dipendenze sincrone profonde in favore di un paradigma *Event-Driven* (Publish/Subscribe). Questa scelta riduce drasticamente il rischio di *Cascading Failures*, sebbene introduca un *overhead* di latenza dovuto ai molteplici *hop* di rete tra i microservizi. Le chiamate verso l'IA Agentica sono protette da policy di **fail-open**: in caso di latenza eccessiva o *outage* del Bridge Service, l'incrocio degrada verso logiche deterministiche locali, garantendo la sicurezza fisica indipendentemente dallo stato del livello cognitivo.
- **Asse Z (Partizionamento e Confinamento del Danno):** L'architettura implementa il partizionamento dei dati e delle responsabilità per minimizzare il **Blast Radius** (raggio d'impatto) di eventuali guasti. A livello *Edge*, ogni nodo incrocio opera in isolamento fisico, gestendo esclusivamente la propria direttrice. A livello cognitivo, SRS adotta un'architettura **Cell-based** (Cella A e Cella B isolate). Sebbene questo "isolamento" possa limitare l'ottimizzazione strategica globale, esso garantisce che un *kernel panic* o un'allucinazione dell'IA in una cella rimangano confinati, permettendo al Bridge Service di dirottare il traffico sulla cella superstita senza interruzioni sistemiche.

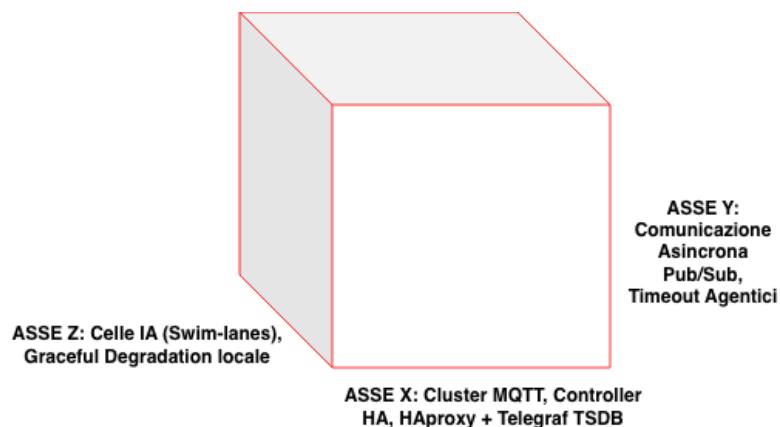


Figura 2.1: Mappatura dell'architettura SRS sull'Availability Cube

Proprio seguendo questo approccio difensivo stratificato e mappato sulle dimensioni del Cubo, l'infrastruttura di base dello Smart Road System (SRS) è stata concepita per eliminare ogni singolo punto di fallimento (SPoF) nel viaggio che il dato compie dalla strada fino al livello di controllo. Di seguito, l'architettura viene analizzata nel dettaglio attraverso i suoi quattro livelli logici, strettamente interconnessi ma tecnologicamente disaccoppiati.

2.2 L'Edge Layer e la Graceful Degradation

Il viaggio del dato inizia all'Edge Layer composto dai "Nodi Incrocio". Ogni nodo è un microservizio autonomo: all'avvio, interroga la propria configurazione per dedurre autonomamente le direttrici da gestire e i nodi adiacenti a cui è collegato, eliminando la necessità di un coordinatore centrale per l'inizializzazione. Essendo questo un sistema Safety Critical, i guasti a questo livello mettono a rischio l'incolumità umana.

Per mitigare guasti hardware o crash del software locale, l'architettura applica in modo rigoroso il principio della Graceful Degradation. Se un singolo semaforo dell'incrocio subisce un'avaria, il nodo non va in blocco totale. Al contrario, isola il componente guasto (portandolo offline) e fa transitare tutte le altre direttrici in uno stato di emergenza a "giallo lampeggiante". Il sistema rinuncia all'ottimizzazione intelligente dei flussi degradando le proprie performance, ma mantiene un livello minimo di operatività che garantisce la sicurezza vitale dell'incrocio.

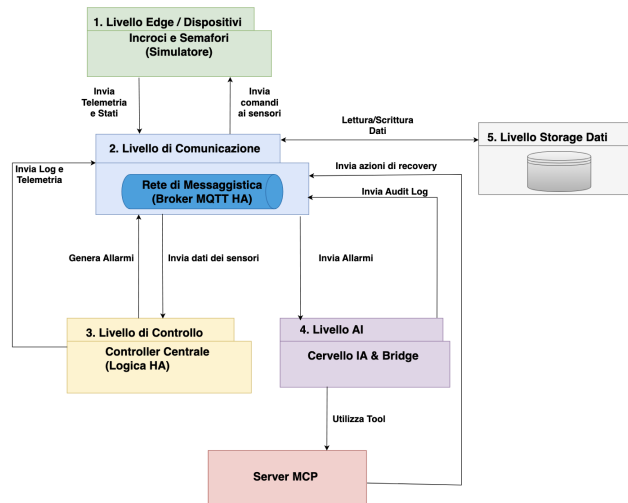


Figura 2.2: Architettura completa

2.3 Il Communication Layer

I nodi fisici non comunicano mai in modo sincrono con il cervello centrale poiché le chiamate sincrone tra domini differenti rappresentano un anti-pattern che genera colli di bottiglia e propagazione dei guasti. La comunicazione avviene in modo asincrono tramite un Communication Layer basato sul paradigma Publish/Subscribe.

Per garantire la massima disponibilità, il livello di rete applica il principio della clonazione dei componenti e della tolleranza ai guasti. Il broker di messaggistica non è un'istanza singola, ma un cluster Active-Active composto da due nodi EMQX che sincronizzano costantemente lo stato e le tabelle di routing. Per proteggere questo cluster e smistare il carico, sono state anteposte due istanze di bilanciatori HAProxy operanti a Livello 4 (TCP) dotati di monitoraggio attivo (Health Checks) per dirottare il traffico dai nodi guasti. Tutti i client del sistema implementano un failover automatico tra i due bilanciatori, garantendo un'infrastruttura di rete totalmente priva di SPoF.

Una sfida critica delle reti distribuite è la gestione dei blackout. Se l'intera rete cadesse, al momento del ripristino centinaia di semafori tenterebbero di riconnettersi simultaneamente. Per prevenire questa congestione, la logica di riconnessione dei client implementa un algoritmo matematico di backoff randomizzato (Jitter): le riconnessioni subiscono un ritardo casuale calcolato dinamicamente, "spalmando" il carico sul broker in un intervallo di tempo gestibile e garantendo una Resilience fluida.

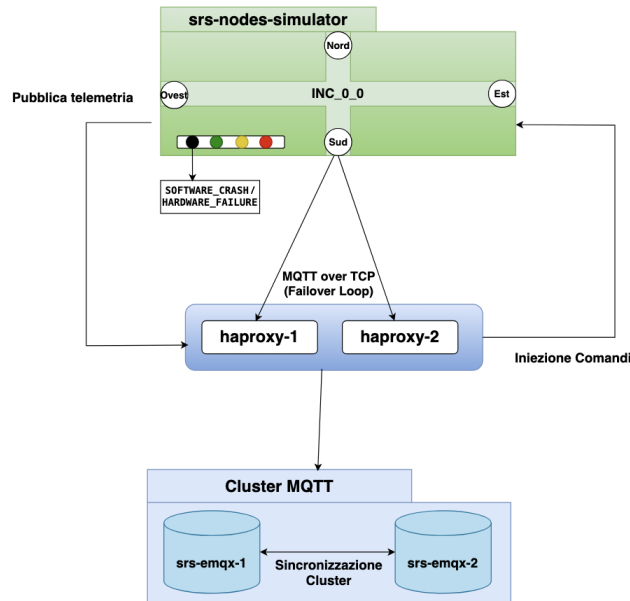


Figura 2.3: Communication Layer

2.4 Il Persistence Layer, Disaccoppiamento e Resilienza Infrastrutturale

La telemetria prodotta dai nodi deve essere storicizzata per l'analisi e per fornire memoria episodica all'Intelligenza Artificiale. A tal fine, il sistema adotta InfluxDB, un Time-Series Database ottimizzato per flussi continui, implementato in architettura Dual-Node (Alpha e Beta).

Nelle fasi iniziali di progettazione, la mancanza di un clustering nativo in InfluxDB aveva imposto un approccio di *Dual-Write* a livello applicativo. Tuttavia, costringere i worker a gestire connessioni HTTP dirette verso i database introduceva latenze inaccettabili, legando le performance del livello cognitivo alla velocità dei dischi fisici (Anti-Pattern).

L'architettura definitiva risolve questo collo di bottiglia evolvendo verso un disaccoppiamento totale. L'applicazione non comunica più con il database, ma applica il paradigma *Fire-and-Forget*: converte i dati nel formato nativo (Line Protocol) e li pubblica sul cluster MQTT con Quality of Service 1 (garanzia di consegna). La scrittura fisica sui due nodi InfluxDB è demandata a un cluster di agenti **Telegraf** che agiscono in modalità *Active-Active* tramite *Shared Subscriptions* MQTT.

Questa evoluzione sposta radicalmente i trade-off di consistenza:

- **Zero-Latency per l'Applicazione:** I worker dell'IA non subiscono più alcun blocco di rete; la scrittura appare istantanea.
- **Eventual Consistency Assoluta:** Accettiamo che i database InfluxDB presentino una *Weak Consistency* temporanea. Tuttavia, anche in caso di caduta di rete tra i DB e il resto del sistema, gli agenti Telegraf utilizzano buffer su disco nativi per trattenere la telemetria, riversandola senza alcuna perdita non appena i nodi tornano online.

Parallelamente, il failover in lettura è stato astratto dal codice: i controller interrogano un bilanciatore HAProxy dedicato che maschera lo stato di salute dei database, instradando le query IA solo verso il nodo InfluxDB sano.

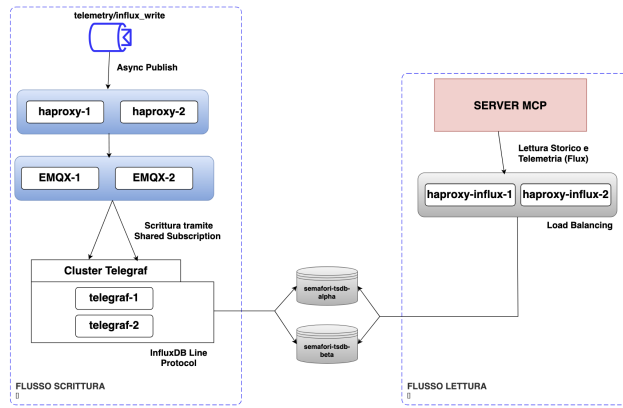


Figura 2.4: Persistence Layer

2.5 Il Control Layer: Watchdog, Quorum e Prevenzione Split-Brain

Se per la telemetria l'*Eventual Consistency* (garantita dal disaccoppiamento asincrono e dai buffer di Telegraf) è un compromesso architetturale ideale, a livello decisionale le regole cambiano radicalmente. Il Main Controller funge da "Watchdog" deterministico, aggregando i dati per identificare guasti fisici e ingorghi. Poiché deve lanciare allarmi ufficiali che modificheranno lo stato dell'incrocio, non può esserci ambiguità: è necessaria una *Strong Consistency*.

Per eliminare il SPoF senza creare conflitti, il Controller opera all'interno di un cluster in modalità Active/Standby. L'elezione del nodo Leader è garantita tramite un algoritmo di Distributed Quorum (Redlock) poggato su un cluster esterno di tre nodi Redis. Solo il controller che ottiene la maggioranza (2 voti su 3) diviene il Leader, con un "lease" temporale (TTL) di pochi secondi.

Il rischio più insidioso di questa topologia è lo Split-Brain: a causa di una partizione di rete, due controller potrebbero non vedersi e dichiararsi entrambi Leader, inviando comandi contrastanti ai semafori. Per neutralizzare questa grave vulnerabilità, il sistema affianca al Quorum un monitoraggio tramite Heartbeat (battito cardiaco) su MQTT: se un controller rileva la presenza di un altro Leader legittimo operante, la logica di sistema lo costringe ad "abdicare" istantaneamente, tornando allo stato passivo e preservando così l'assoluta coerenza delle operazioni nel mondo fisico.

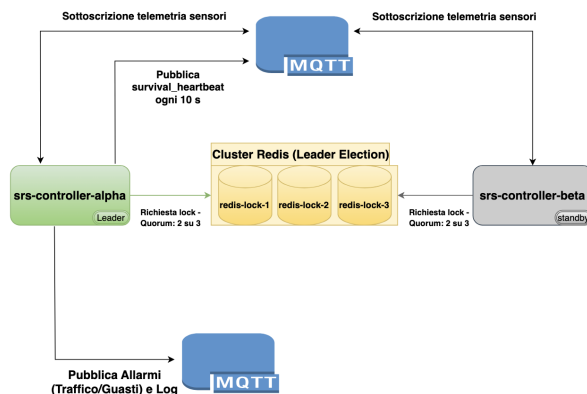


Figura 2.5: Control Layer

Capitolo 3

Architettura IA: Livello Agentico, Protocollo MCP e Osservabilità

Una volta che il Main Controller ha aggregato i dati e identificato con assoluta certezza un'anomalia, si apre la sfida operativa più complessa: diagnosticare la causa radice e applicare una soluzione in un ambiente incerto. Per superare i limiti dei tradizionali script di automazione che procedono attraverso rigide sequenze di azioni pre-impostate, SRS integra un livello di Agentico IA. A differenza di una semplice automazione, un sistema agentico è in grado di autovalutare la situazione, pianificare iterativamente e adattare la sequenza di azioni in risposta a contesti dinamici.

3.1 Il Bridge Service e la Sincronizzazione Distribuita

Il passaggio delle consegne tra il mondo deterministico e l'Intelligenza Artificiale avviene tramite il Bridge Service, che funge da sistema nervoso centrale per l'instradamento degli allarmi. Per tollerare carichi di lavoro massivi ed evitare colli di bottiglia, il Bridge non è un'istanza singola, ma è scalato orizzontalmente con molteplici Worker in ascolto. Il bilanciamento del carico nativo (Load Balancing) è garantito dall'uso di Shared Subscriptions sul broker MQTT, che smistano gli allarmi in logica Round-Robin. Tuttavia, avere molteplici worker indipendenti introduce un rischio critico per i sistemi distribuiti: la Race Condition. Due worker potrebbero tentare di risolvere lo stesso allarme contemporaneamente, sovrapponendo i comandi. Per neutralizzare questa minaccia il sistema implementa un meccanismo di Distributed Locking su Redis: prima di ingaggiare l'IA, il worker deve acquisire un "lucchetto" esclusivo sull'incrocio. Un processo in background (**Lock Refresher**) rinnova costantemente questo blocco finché l'intervento non è concluso, garantendo che le operazioni siano mutualmente esclusive.

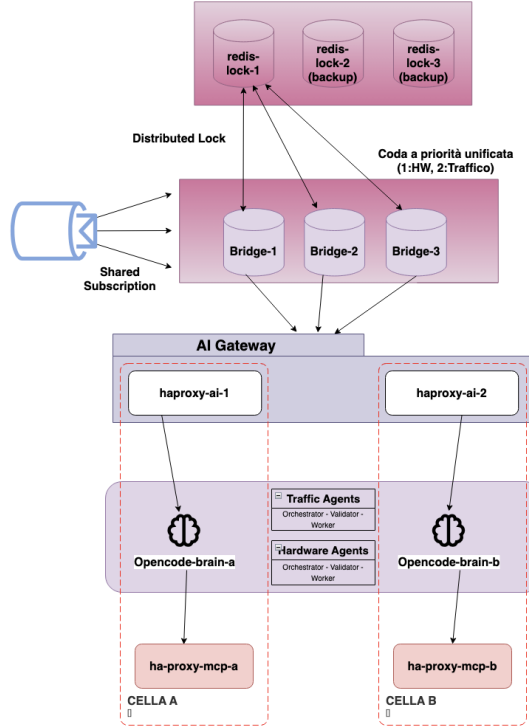


Figura 3.1: Bridge Service e Livello IA

3.2 Z-Axis Scalability e l'Isolamento dell'IA

L'integrazione di modelli linguistici di grandi dimensioni (LLM) introduce un grado di imprevedibilità strutturale: i modelli possono subire latenze estreme, andare in timeout o collassare. Per questo motivo, il livello decisionale non è un monolite, ma è frammentato applicando il principio teorico della Z-axis Scalability (partizionamento) e delle Swim-lanes (corsie isolate). L'intelligenza è divisa in "Celle IA" indipendenti (Cella A e Cella B) all'interno di un'architettura a compartimenti stagni. Ogni cella opera in modalità Active-Active e contiene il proprio bilanciatore, il proprio motore di ragionamento (OpenCode Brain) e il collegamento con gli attuatori. Se la Cella A va in kernel panic o non riesce a comunicare con il modello, il raggio dell'esplosione rimane rigidamente confinato a quello specifico scompartimento. Il Bridge Service intercetta l'errore ed esegue istantaneamente un failover, dirottando il traffico verso la Cella B senza interrompere il servizio globale.

3.2.1 Opencode Brain e Architettura Gerarchica degli Agenti

L'orchestrazione intelligente del sistema SRS non è affidata a un singolo Large Language Model (LLM) generalista, ma a una complessa architettura agantica implementata tramite l'ambiente runtime **Opencode Brain**. Al fine di rispettare il principio del Minimo Privilegio (*Least Privilege*) e per prevenire il rischio di *Agent Drift*, il sistema adotta un modello decisionale gerarchico suddiviso tra Agenti Primari (*Primary*) e Sub-Agenti (*Subagent*).

- **Agenti Primari (Orchestratori):** Nodi come **hardware-orchestrator** e **traffic-orchestrator** fungono esclusivamente da *router* decisionali. Non eseguono calcoli diretti e non hanno accesso agli attuatori fisici. Il loro *System Prompt* li vincola a una rigida *Policy-as-Code* ("*DELEGATE ONLY*"), delegando l'esecuzione ai sub-agenti e gestendone le eventuali eccezioni.
- **Sub-Agenti (Esecutori Specializzati):** Agenti come **traffic-validator** e **traffic-worker** possiedono un contesto cognitivo (*Context Window*) estremamente limitato e mirato a un singolo *task*.

Questa segmentazione delle responsabilità funge da **Bulkhead logico**. Se il **traffic-worker** subisce un'allucinazione durante il calcolo dei cicli semaforici, l'errore non compromette il **traffic-orchestrator**, che può intercettare l'anomalia e forzare un *Safe Fallback* (es. degradazione a controllo umano).

3.2.2 Guardrails e Model Context Protocol (MCP)

Per ingabbiare l'LLM probabilistico in un perimetro di sicurezza deterministico, l'architettura sfrutta il **Model Context Protocol (MCP)**. Gli agenti non modificano direttamente lo stato del sistema stradale, ma sono obbligati a interagire tramite *tools* rigidamente tipizzati esposti dall'MCP Server.

Ad esempio, la validazione del traffico è blindata: il **traffic-validator** deve chiamare il tool **green_duration_check_loc**. Il tool restituisce un formato predeterminato (**STATUS|reason=X|cars=Y|ts=Z**) forzando l'LLM a elaborare *dati fattuali* (Memoria Episodica fornita in input) piuttosto che affidarsi alla propria conoscenza statistica latente. Le istruzioni **CRITICAL** (es. *"If tool fails, output: ERROR: MCP_UNAVAILABLE"*) agiscono come **Circuit Breakers applicativi**, interrompendo immediatamente l'inferenza in caso di latenze di rete.

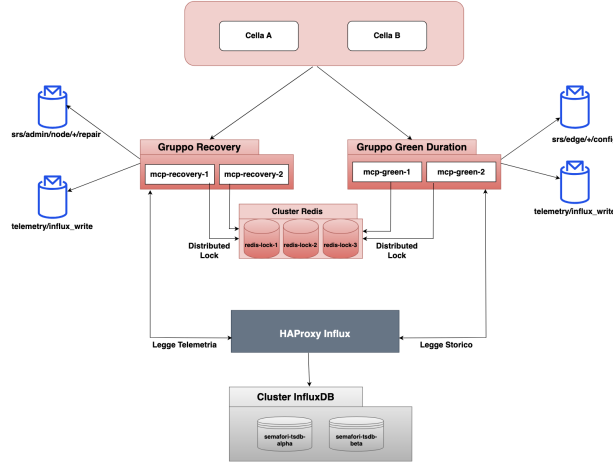


Figura 3.2: Livello MCP

3.2.3 Workflow del Traffico e Recovery Hardware (Sense-Think-Act)

I workflow implementati incarnano il ciclo chiuso di controllo (*Sense-Think-Act*) tipico dell'IA Agentica.

- **Hardware Recovery Workflow:** In caso di guasto periferico, l'**hardware-orchestrator** delega al **policy-validator-agent** l'analisi telemetrica. Solo se la validazione ha successo, il **recovery-agent** viene autorizzato a richiamare i tool MCP di riavvio (**edge_recovery_execute_node_restart**). In caso di fallimento o *lock* tecnico, il prompt impone l'azione **RETRY** o la *Graceful Degradation* tramite **ESCALATION** umana.
- **Traffic Optimization Workflow:** Il **traffic-worker** esegue ottimizzazioni basate su vincoli logico-matematici dichiarati nel prompt (es. Max 3 cicli, max 40 secondi, threshold di 15 auto). Questo vincola l'LLM a distribuire i verdi su cicli brevi, privilegiando lo smaltimento fluido rispetto a un'unica e rischiosa apertura prolungata.

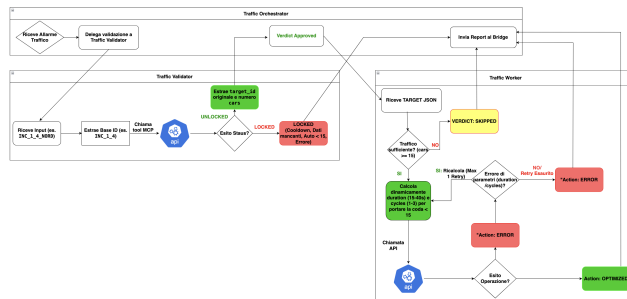


Figura 3.3: Workflow Traffic Agent

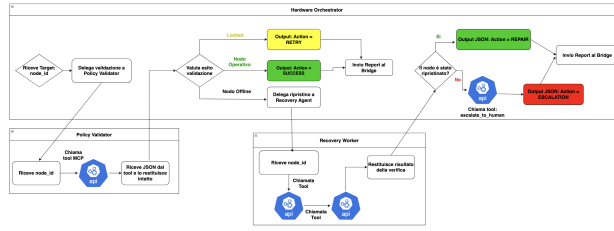


Figura 3.4: Workflow Hardware Agent

3.2.4 Protezione OPEX: Il parametro *Steps* e la Verbosity

L'integrazione di LLM aziendali espone il sistema a vulnerabilità economiche in caso di loop infiniti (*Infinite Generation*). Per mitigare questo rischio e ottimizzare il *Return on Agent* (ROA), la configurazione `opencode.json` introduce due parametri fondamentali:

1. **Parametro `steps`:** Ogni agente possiede un *hard-limit* di iterazioni permesse per completare il proprio task (es. 10 per `hardware-orchestrator`, 6 per `policy-validator`). Questo limite (*Bounded Execution*) funge da **Cost Ceiling meccanico**. Se un agente entra in un loop di allucinazione tentando di riprovare un tool guasto, l'esecuzione viene terminata forzatamente raggiunto il limite di step, impedendo l'erosione del budget OPEX.
2. **Parametro `textVerbosity`:** `"low"`: Istruendo l'ambiente a sopprimere le motivazioni conversazionali e vincolando l'output a formati JSON stringenti (*"Return EXACTLY the JSON received"*), il sistema minimizza drasticamente i *Token di Output* (i più costosi nella fatturazione API), massimizzando l'efficienza economica del triage.

3.3 Osservabilità e Asimmetria nei Trade-off Economici (80/20 Rule)

L'ultimo strato dell'architettura è l'Observability Plane, essenziale per la tracciabilità delle azioni agentiche (Audit Logging) e la visualizzazione delle metriche transazionali. Attraverso strumenti come Grafana, Portainer e Dozzle, gli operatori possono ispezionare le code di traffico e verificare la logica che ha spinto l'IA a prendere specifiche decisioni operative. In questa parte si è scelto deliberatamente di non implementare misure anti-SPoF. Se un server Grafana collassa, l'operatore umano perde temporaneamente la visibilità visiva, ma l'impatto sul traffico fisico reale e sulle capacità di rimediazione dell'IA è nullo.

Capitolo 4

Policy di Sicurezza Operativa e Guardrails per l'Intelligenza Artificiale

L'integrazione di sistemi basati su Large Language Models (LLM) all'interno di infrastrutture di controllo fisico introduce un cambio di paradigma rispetto al software deterministico tradizionale. Gli agenti IA, essendo sistemi adattivi e probabilistici, introducono una nuova classe di vulnerabilità: allucinazioni, escalation dei privilegi ed esecuzione di comandi non sicuri. Per questo motivo, l'intero livello cognitivo di Smart Road System (SRS) è stato ingegnerizzato seguendo il paradigma del Secure-by-Design, trasformando la fiducia nell'IA in un processo matematicamente misurabile e limitato.

4.1 Principio del Least Privilege e Bounded Execution

In SRS, l'agente IA è concepito esclusivamente come un Operational Assistant e non come un controllore con poteri illimitati sull'infrastruttura. Per mitigare il rischio di compromissioni o azioni distruttive, il sistema applica rigorosamente il Principle of Least Privilege.

A livello pratico, il motore cognitivo (OpenCode Brain) non possiede alcuna credenziale per accedere direttamente al database InfluxDB o per pubblicare liberamente sul broker MQTT. Qualsiasi interazione con il mondo fisico deve transitare attraverso i server MCP, che fungono da perimetro di sicurezza. All'interno di questi server, gli strumenti messi a disposizione dell'IA rispettano il principio della Bounded Execution. L'agente non possiede strumenti generici, ma solo small, focused, bounded tools. Ad esempio, lo strumento di ottimizzazione semaforica (`green_duration_optimize_node`) contiene un limite rigido: non accetta parametri che assegnino un verde superiore a 40 secondi o un numero di cicli superiore a 3. Qualsiasi tentativo dell'IA di generare (o allucinare) parametri al di fuori di questo dominio viene respinto deterministicamente a livello MCP.

4.2 Policy-as-Code e Loop Prevention

La resilienza del sistema dipende da comportamenti verificabili piuttosto che da output apparentemente plausibili generati dall'IA. Per questo motivo, prima di eseguire azioni ad alto impatto, SRS obbliga l'agente a un processo di validazione basato su strumenti Read-only by default.

Nel flusso di ripristino di un nodo guasto (Recovery), l'agente orchestratore non può inviare un comando di riavvio basandosi solo su una propria deduzione, ma è obbligato a interrogare prima il sub-agente `policy-validator-agent` che esegue lo strumento di diagnostica in sola lettura `validate_node_for_restart`. Questo strumento interroga lo storico operativo (Audit Log) applicando una logica di Policy-as-Code: se rileva che l'incrocio è già stato riavviato negli ultimi 5 minuti, restituisce un verdetto di blocco (BLOCKED). Questa architettura implementa un meccanismo vitale di Loop Prevention: impedisce che, di fronte a un guasto hardware irreparabile sul campo, l'IA consumi inutilmente risorse (token) o saturi la rete entrando in un ciclo infinito di tentativi di riavvio a vuoto.

4.3 Mitigazione dell'Agent Drift e delle Allucinazioni

Gli LLM sono progettati per ottimizzare la coerenza statistica del testo piuttosto che la verità fattuale. Nel tempo, questo può portare all'Agent Drift, ovvero una deviazione graduale e imprevedibile del comportamento dell'agente rispetto agli obiettivi previsti.

Per contrastare questo fenomeno, i prompt del sistema SRS fungono da ulteriore livello difensivo. Nei prompt di sistema dei Micro-Agenti sono stati iniettati Guardrails espliciti sotto forma di regole invalicabili ("Specify

Do” e “Specify Don’t”). Ad esempio, la **CRITICAL ANTI-HALLUCINATION RULE** impone testualmente: “**You MUST NOT generate the 'final_report' JSON immediately. Never hallucinate data. Wait for subagent tools to finish**”. Questa istruzione costringe l’orchestratore a vincolare il proprio ragionamento probabilistico esclusivamente all’output deterministico fornito dai tool, riducendo drasticamente la superficie per le allucinazioni.

4.4 Operational Safety Policy e Gestione dell’Ambiguità

SRS implementa una chiara Operational Safety Policy accettando che le capacità di rimediazione autonoma debba avere dei limiti precisi.

Se il contesto operativo non è chiaro, o se il livello di confidenza dell’IA nella risoluzione del problema è basso, la policy di SRS vieta l’esecuzione di azioni speculative. In queste situazioni di ambiguità, il sistema applica una logica di Safe Fallback:

1. L’agente IA interrompe l’intervento e passa la richiesta allo stato di Escalation (deferendo l’intervento a un operatore umano).
2. Parallelamente, l’Edge Layer ai semafori attiva le proprie logiche di Graceful Degradation locali (passaggio al giallo lampeggiante), rinunciando all’ottimizzazione globale ma garantendo la messa in sicurezza immediata e reversibile dell’incrocio.

Infine, per garantire la massima Accountability, tutte le decisioni finali dell’IA (sia i successi che le escalation) vengono obbligatoriamente registrate nell’Audit Log centrale, specificando non solo l’azione intrapresa, ma anche la motivazione logica (reason) fornita dall’agente, permettendo agli operatori umani una completa revisione post-incidente.

Capitolo 5

Validazione e Testing del Sistema

La validazione di un'architettura *Safety Critical* e distribuita come **Smart Road System (SRS)** richiede un approccio multidimensionale che superi il semplice collaudo funzionale. In linea con i paradigmi della resilienza, il sistema è stato sottoposto a campagne di test mirate a identificare i colli di bottiglia (*Bottlenecks*), validare le strategie di *Failover* e certificare la sostenibilità economica degli agenti IA.

5.1 Metodologia e Strategia di Testing

Il piano di collaudo è stato suddiviso in tre macro-obiettivi, seguendo la distinzione accademica tra sistemi complessi e complicati:

- **Benchmarking e Stress Testing:** Determinazione dei limiti fisici del *Communication Layer* in condizioni di carico estremo.
- **Chaos Engineering:** Verifica della continuità operativa a fronte di fallimenti improvvisi di nodi infrastrutturali.
- **Endurance e Validation Testing:** Monitoraggio della stabilità a lungo termine e della corretta gestione della concorrenza nell'orchestrazione agentica.

5.2 Stress Test del Communication Layer (MQTT Flooding)

Per individuare il punto di rottura del livello di rete è stato condotto uno **Stress Test** massivo utilizzando script di *flooding* multi-processo. L'obiettivo era saturare il bilanciatore HAProxy e il cluster EMQX per osservare il comportamento del sistema in scenari di congestione telemetrica anomala.

5.2.1 Risultati Empirici a 10.000 msg/sec

Durante il picco di carico, con un'iniezione costante di 10.000 messaggi al secondo (QoS 0), le metriche estratte tramite `docker stats` hanno confermato l'estrema efficienza della scelta architetturale.

Componente	CPU %	Memoria (MiB)	Network I/O (MB)
srs-emqx-1	14.40%	238.9	199.0 / 20.3
srs-emqx-2	10.06%	233.5	193.0 / 5.28
srs-haproxy-1	1.02%	13.7	397.0 / 397.0

Tabella 5.1: Metriche di assorbimento risorse con carico iniettato di 10.000 msg/sec.

L'analisi dei dati evidenzia che HAProxy gestisce quasi 400 MB di traffico aggregato con un impegno computazionale trascurabile ($\sim 1\%$), confermando che il bilanciamento a Livello 4 (TCP) è la soluzione ottimale per minimizzare l'overhead. I broker EMQX mostrano un ampio margine di *Capacity Planning*, operando ben al di sotto della soglia di saturazione.

5.2.2 Analisi Comparativa: Il Costo della Certezza (QoS 0 vs QoS 1)

Nella teoria dei sistemi distribuiti, ogni garanzia di affidabilità comporta un costo in termini di risorse e latenza. Per quantificare empiricamente questo assioma, lo stress test a 10.000 msg/sec è stato ripetuto elevando la *Quality of Service* del protocollo MQTT da **QoS 0** (*At most once*) a **QoS 1** (*At least once delivery*).

L'introduzione del QoS 1 impone al broker di inviare una conferma di ricezione (PUBACK) per ogni singolo pacchetto, trasformando una comunicazione unidirezionale in un *handshake* bidirezionale. I risultati estratti da `docker stats` evidenziano chiaramente questo impatto:

Componente	CPU %	Memoria (MiB)	Network I/O (MB)
srs-emqx-1	22.69%	228.2	61.3 / 20.2
srs-emqx-2	16.81%	231.3	61.2 / 20.0
srs-haproxy-1	11.43%	13.7	158.0 / 151.0

Tabella 5.2: Metriche di assorbimento risorse a 10.000 msg/sec con QoS 1.

Il confronto tra i dati in Tabella 5.1 e Tabella 5.2 permette di delineare le seguenti considerazioni architetturali:

- **L'Esplosione dell'Overhead sul Livello di Trasporto:** Il dato più eclatante riguarda `srs-haproxy-1`. Il consumo di CPU è decuplicato, passando dall'1.02% all'11.43%. Inoltre, il traffico di rete sul proxy mostra una natura simmetrica (158MB In / 151MB Out), prova evidente dell'enorme mole di PUBACK che transitano in risposta verso i flooder.
- **Impatto sullo State Management (Broker CPU):** I nodi EMQX registrano un incremento del carico computazionale di circa il 60% (dal 10-14% al 16-22%). A differenza del QoS 0, il broker deve ora mantenere lo stato del messaggio in memoria fino al completamento dell'*handshake*. Tuttavia, la memoria RAM rimane eccezionalmente stabile (~230 MiB), indicando che il sistema smaltisce gli ACK in tempo reale senza subire *Queue Overflow*.
- **Considerazioni Ingegneristiche sui Trade-off:** Il test dimostra che, sebbene l'infrastruttura sia in grado di sostenere il QoS 1, l'adozione di tale livello per la telemetria grezza ad alta frequenza è un anti-pattern. Accettare una tolleranza alla perdita di pacchetti (QoS 0) per l'ingestione massiva dai sensori stradali è una scelta obbligata per garantire la scalabilità. Al contrario, il QoS 1 è strettamente riservato al Control Plane e al Persistence Layer (comandi di configurazione, decisioni dell'IA e messaggi dei worker verso i buffer di Telegraf per il salvataggio su database), dove l'assoluta affidabilità della consegna giustifica l'incremento di overhead.

5.3 Chaos Engineering: Validazione dell'Immunità ai Guasti

La resilienza del sistema SRS non è stata solo teorizzata, ma verificata attraverso una campagna di **Chaos Engineering** sistematica. L'obiettivo è stato quello di indurre il fallimento forzato di ogni singolo nodo dell'infrastruttura (strategia di "esplosione" controllata) per dimostrare che il sistema è in grado di operare senza interruzioni indipendentemente dal punto di guasto.

5.3.1 Matrice dei Test di Fallimento (Node Failure Matrix)

Di seguito vengono elencati i test eseguiti e la relativa risposta architetturale, confermando che nessun singolo punto di fallimento (*SPoF*) è in grado di abbattere il servizio:

1. Livello di Comunicazione (Broker MQTT):

- **Azione:** Terminazione forzata del container `srs-emqx-1`.
- **Risultato:** Il cluster EMQX ha mantenuto la topologia tramite il nodo superstite. I client sono stati reindirizzati istantaneamente dal bilanciatore Haproxy senza perdita di messaggi pendenti.

2. Livello di Comunicazione (Haproxy):

- **Azione:** Terminazione forzata del container `srs-haproxy-1`.
- **Risultato:** Il sistema si disconnette e ogni singolo nodo effettua un connection failover per collegarsi ad haproxy-2 e tornare alla normale esecuzione.

3. Livello di Persistenza (Database TSDB):

- **Azione:** Spegnimento improvviso del nodo `semafori-tsdb-alpha`.

- **Risultato:** Zero impatto sul codice applicativo (nessuna eccezione o timeout generato nei worker). In lettura, `haproxy-influx` ha rilevato il fallimento dell'health-check e ha dirottato istantaneamente tutte le letture dell'IA sul nodo Beta. In scrittura, i nodi Telegraf hanno continuato a salvare i dati sul nodo Beta superstite, mettendo automaticamente in buffer i pacchetti destinati ad Alpha fino al suo riavvio. Nessun dato telemetrico o log di audit è andato perso.

4. Livello di Controllo (Leadership):

- *Azione:* Crash del Main Controller attualmente in stato di *Leader*.
- *Risultato:* La scadenza del TTL sul cluster Redis ha innescato una nuova elezione tramite l'algoritmo **Redlock**. Una delle istanze in *Follower* ha acquisito il lock distribuito entro 10 secondi, ripristinando le funzioni di *Watchdog* hardware e traffico.

5. Livello Decisionale (Celle IA):

- *Azione:* Inattivazione della Cella IA *Alpha* (`srs-opencode-a`).
- *Risultato:* Il Bridge Service, rilevando l'errore di connessione (500/Timeout), ha applicato il principio di **Bulkheading**. Le richieste sono state ridirezionate sulla Cella *Beta*, dimostrando che il fallimento di un intero ambiente di inferenza non paralizza l'orchestrazione intelligente.

6. Livello di Ingress (Load Balancer):

- *Azione:* Terminazione forzata di `srs-haproxy-1`.
- *Risultato:* Sfruttando la configurazione a doppia porta e doppio bilanciatore, i client (nodi edge e controller) hanno eseguito il *failover* sulla porta del secondo bilanciatore, validando la ridondanza totale anche nel punto di accesso alla rete.

5.3.2 MTTR: Componente vs. Sistema

L'analisi dei tempi di ripristino evidenzia una distinzione fondamentale tra le metriche di affidabilità locali e globali. In architetture monolitiche tradizionali, il guasto di un nodo si propaga a cascata, rendendo il *Mean Time To Recovery* (MTTR) del componente coincidente con il MTTR del sistema.

Nel progetto SRS, l'applicazione rigorosa dei pattern di *Fault Tolerance* ha permesso di disaccoppiare queste due metriche. I test di iniezione di guasti hanno registrato un **MTTR a livello di componente** quantificabile in pochi secondi (es. ~5s per il failover di rete MQTT, ~10s per l'elezione di un nuovo Main Controller tramite Redlock).

Tuttavia, grazie alla ridondanza Attivo-Attivo e alle logiche di instradamento trasparente (HAProxy), questi fallimenti locali vengono mascherati all'utente finale e agli attuatori fisici. Di conseguenza, il **MTTR a livello di sistema** risulta virtualmente nullo (**Zero Downtime**). Il servizio di orchestrazione del traffico non subisce interruzioni, dimostrando che l'infrastruttura non solo è in grado di ripararsi autonomamente (*Self-Healing*), ma lo fa in modo del tutto trasparente rispetto all'operatività *Safety Critical* dell'ambiente urbano.

5.3.3 Mean Time To Detect (MTTD) e il Trade-off del Watchdog

In un'architettura decentralizzata basata su scambi di messaggi asincroni, la rilevazione di un guasto non è un evento istantaneo. La metrica del **Mean Time To Detect** (MTTD) misura il ritardo fisiologico tra il verificarsi dell'anomalia e la presa di consapevolezza da parte del sistema di monitoraggio.

Nel progetto SRS, il MTTD è stato implementato tramite il pattern **Watchdog** centralizzato sul Main Controller. I nodi edge inviano un *Heartbeat* telemetrico a intervalli regolari (10 secondi). Il Controller valuta lo stato di salute basandosi sul parametro `HARDWARE_CHECK_INTERVAL`, impostato empiricamente a 30 secondi. Di conseguenza, il MTTD del sistema per i guasti periferici è matematicamente confinato in una finestra compresa tra i 30 e i 40 secondi.

- **Il Trade-off dei Falsi Positivi (Network Jitter):** La scelta di un MTTD di ~30 secondi rappresenta un compromesso deliberato. Impostare un *timeout* più aggressivo ridurrebbe il MTTD, ma esporrebbe il sistema a un alto tasso di *Falsi Positivi* causati dalla normale latenza di rete (*Network Jitter*).
- **Impatto sull'OPEX:** In un'architettura che integra un'Intelligenza Artificiale Agentica, ogni falso positivo comporterebbe l'invio di un allarme al Bridge Service e l'attivazione inutile dell'LLM per tentare una rimediazione. Accettare un MTTD leggermente superiore funge da *Debounce* architetturale, filtrando i disturbi di rete transitori e proteggendo rigorosamente il budget operativo da chiamate API superflue.

Capitolo 6

Analisi Economica (OPEX), Trade-off Ingegneristici e Return on Agent (ROA)

6.1 Analisi del Payload e Return on Agent (ROA)

L'implementazione di un livello agentico introduce nuovi livelli di complessità e costi operativi. Valutare il *Return on Agent* (ROA) significa misurare analiticamente se l'automazione intelligente convenga economicamente rispetto all'intervento costante di un operatore umano L1 (*First-Level Support*).

Per operare in modo sicuro e contestualizzato, l'agente basato su *Model Context Protocol* richiede un'elevata densità informativa. Per prendere una singola decisione (es. ottimizzare un asse semaforico), l'LLM deve processare un blocco di contesto considerevole: il *System Prompt* (inclusivo dei *guardrails* di sicurezza), le rigide definizioni degli schemi JSON per l'uso dei tool e lo storico della telemetria. Le misurazioni empiriche sul sistema SRS hanno verificato che questo genera richieste fortemente asimmetriche: il *payload* di input si attesta stabilmente tra gli 8.000 e i 10.000 token, mentre l'output (la decisione operativa strutturata) si concentra in circa 100-150 token.

Basandoci su un approccio pessimistico (*Worst-Case Scenario*) e adottando i listini dei principali LLM di classe Enterprise (es. \$1,25 per 1M token in input e \$5,00 per 1M token in output), possiamo calcolare il costo vivo reale:

- **Costo Input:** $(10.000/1.000.000) \times \$1,25 = \$0,0125$
- **Costo Output:** $(100/1.000.000) \times \$5,00 = \$0,0005$
- **Costo totale per operazione:** $\sim \$0,013$ (poco più di 1 centesimo)

Prevedendo un carico operativo intenso pari a circa 100 allarmi di traffico o diagnostici al giorno (3.000 interventi automatizzati al mese), il consumo API puro si assesta a circa \$39,00 al mese.

Se si confronta questa cifra con il costo orario di un operatore umano addetto al monitoraggio del traffico e con il costo sociale di un disservizio, il ROA appare straordinariamente positivo. L'agente permette di automatizzare in sicurezza, 24 ore su 24, tutto il triage di primo livello e di ottimizzare il traffico per meno di 40 dollari al mese, azzerando virtualmente il tempo di reazione (MTTR) a cui un operatore umano sarebbe fisiologicamente soggetto.

6.2 Limiti Economici e Protezione del Budget (Cost Ceiling)

Nonostante il ROA sia eccellente, l'uso di LLM probabilistici espone al rischio di un consumo imprevisto delle risorse, ad esempio in caso di anomalie fisiche latenti sui sensori che provocherebbero continui "falsi positivi" e innescerebbero loop diagnostici infiniti.

Per rispettare le *best-practice* di sostenibilità in cloud e proteggere il budget aziendale (*Circuit Breaker pattern*), SRS implementa un "Agent-related Cost Ceiling" (tetto massimo di spesa) applicato a livello di Proxy API. Il sistema è configurato con un *hard-limit* mensile basato sul traffico nominale (es. impostato a \$100,00, coprendo più del doppio del traffico atteso). Se il consumo raggiunge tale soglia, il proxy inibisce le chiamate all'IA: da quel momento, il sistema degrada in modalità di automazione puramente deterministica, deferendo obbligatoriamente ogni decisione operativa non standard a un cruscotto umano. Questa logica garantisce che i costi dell'Intelligenza Artificiale restino sempre prevedibili e confinati.

Conclusioni e Considerazioni Finali

Il progetto Smart Road System (SRS) dimostra che, in un contesto Safety Critical come quello urbano, l'affidabilità è un pilastro fondamentale della sicurezza pubblica. Le conclusioni del lavoro possono essere riassunte in tre pilastri fondamentali:

- **Ingegneria della Resilienza:** Accettando l'inevitabilità dei guasti in sistemi distribuiti, SRS elimina i singoli punti di fallimento (SPOF) tramite una ridondanza sistematica (Replication). L'uso di cluster MQTT, pipeline di ingestione resilienti (Telegraf in Active-Active), bilanciamento HAProxy per lo storage e quorum distribuito su Redis garantisce un tempo di ripristino (RTO) quasi istantaneo, mentre la Graceful Degradation preserva la sicurezza anche in scenari estremi.
- **Paradigma della "Trusted IA":** L'integrazione dell'IA Agentica non è una delega cieca, ma un processo controllato. Attraverso il protocollo MCP e il principio del minimo privilegio, l'architettura confina l'IA in un perimetro di sicurezza deterministico. Questo approccio neutralizza i rischi di allucinazioni operative, garantendo che ogni azione sia sicura e, in caso di ambiguità, soggetta a escalation umana.
- **Sostenibilità e Valore Economico:** L'analisi del ritorno sull'investimento (ROA) conferma che l'automazione del triage e della diagnostica riduce drasticamente i tempi medi di riparazione (MTTR), rendendo l'IA un asset economicamente giustificato e sostenibile.

In sintesi, SRS incarna la filosofia dei sistemi Mission Critical: non si limita a implementare funzionalità, ma progetta un'infrastruttura capace di adattarsi e restare sicura anche di fronte all'incertezza operativa.